



Welcome to Binary Brains



"Where Knowledge Knows No Limits"



Our Team Members



Sk. Azmain Zunaeid
(Leader)
ID: 251-15-222



Md. Delower Sarker
ID: 251-15-457



Md. Maruf Mia
ID: 251-15-042



Syeda Zawada Farah
ID: 251-15-223



Tahmid Ibne Mofazzol
ID: 251-15-548



SMART STUDENT MARKS ORGANIZER

A modern digital solution for grading system automation.





Course Code: CSE214

Algorithms Lab

Department of Computer Science and Engineering



Daffodil
International
University



Course Instructor



Ms. Nishat Sadaf Lira

Lecturer



Department of CSE



Daffodil International University

Project Management and Team Work



- **Project Leader: Zunaeid** - coordinated project, delegated tasks, managed timeline, verified design, and reviewed final work.
- **Research & Requirements: Farah & Delower** - conducted feasibility study, gap analysis, and gathered references.
- **System Implementation: Tahmid, Delower & Zunaeid** - coded core functions, integrated modules, debugged, and tested functionality.
- **Testing & QA: Maruf** - designed test cases, validated system, ensured robust error handling.
- **Documentation: Zunaeid & Farah** - compiled project report, wrote methodology, structured content.
- **Presentation: Maruf & Tahmid** - created slides, simplified complex details, ensured clarity and visual appeal.

Presentation Outline



- Introduction
- Motivation
- Objectives
- Feasibility Study
- Requirement Analysis

- Implementation
- Results and Discussion
- Advantages and Disadvantages
- Conclusion
- QnA

Introduction

- **Console-based academic record system in C**
- **Demonstrates Quick Sort and Binary Search applications**
- **Two primary modules: Administrative Management & Performance Analysis**
- **Data-driven approach to student progress tracking**



Motivation

- **Pedagogical Motivation**
Bridges the gap between abstract algorithmic theory and practical application
- **Practical Motivation**
Addresses the need for efficient, low-overhead academic record management



Objectives

- Implement persistent record management using linked lists with file-based storage
- Apply **Quick Sort** algorithm to identify and rank top-performing students
- Implement **Binary Search** to provide high-speed student lookups by unique ID
- Develop section-wise performance analysis with visual **bar graph** representations



Feasibility Study

- Technical Feasibility
- Resource Feasibility
- Operational Feasibility
- Algorithmic Feasibility

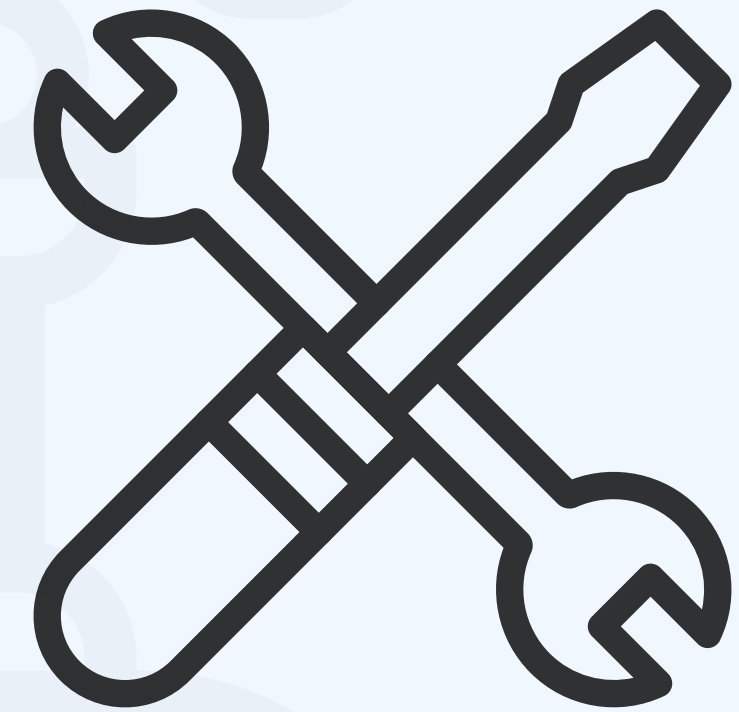


Requirement Analysis

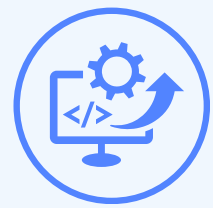


Hardware Requirements

- **Processor:** 1 GHz or higher
- **Memory:** 1 GB RAM minimum
- **Storage:** 10 MB available space
- **Input:** Standard keyboard
- **Output:** Console/terminal display



Requirement Analysis

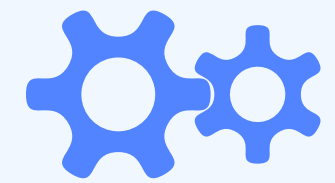


Software Requirements

- **Operating System:** Windows/Linux/macOS
- **Compiler:** GCC, Clang, or any ANSI C compliant compiler
- **Dependencies:** Standard C libraries only

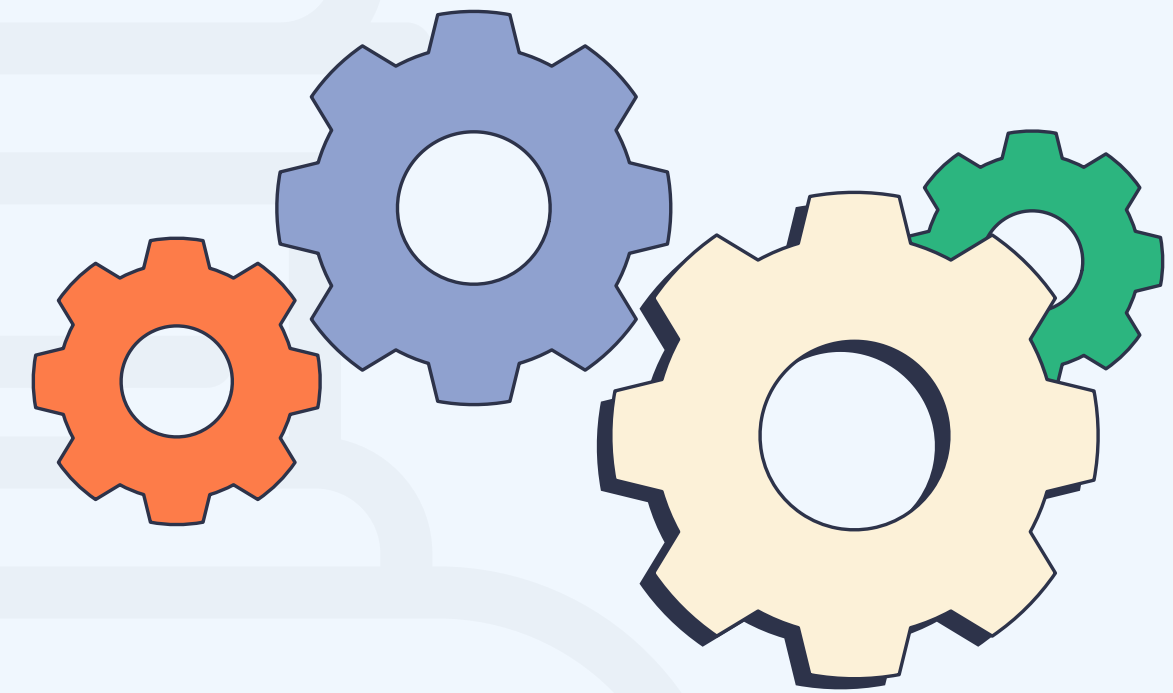


Requirement Analysis



Functional

- Record Management
- Student Ranking
- Advanced Search System
- Performance Analytics
- Data Persistence
- Input Validation

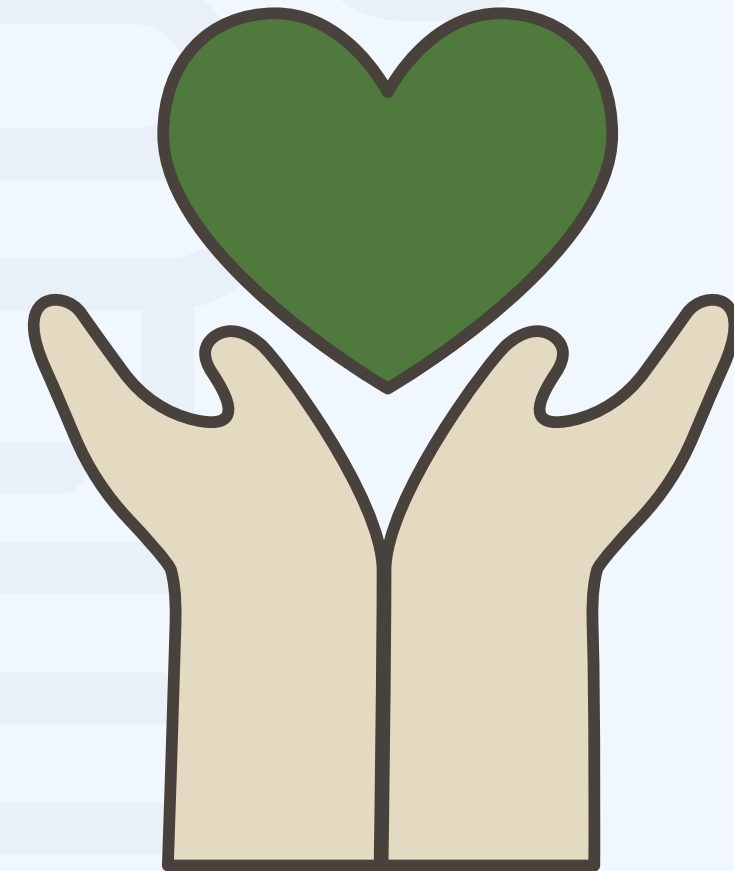


Requirement Analysis



Non-Functional

- Performance
- Reliability
- Usability
- Maintainability
- Memory Safety



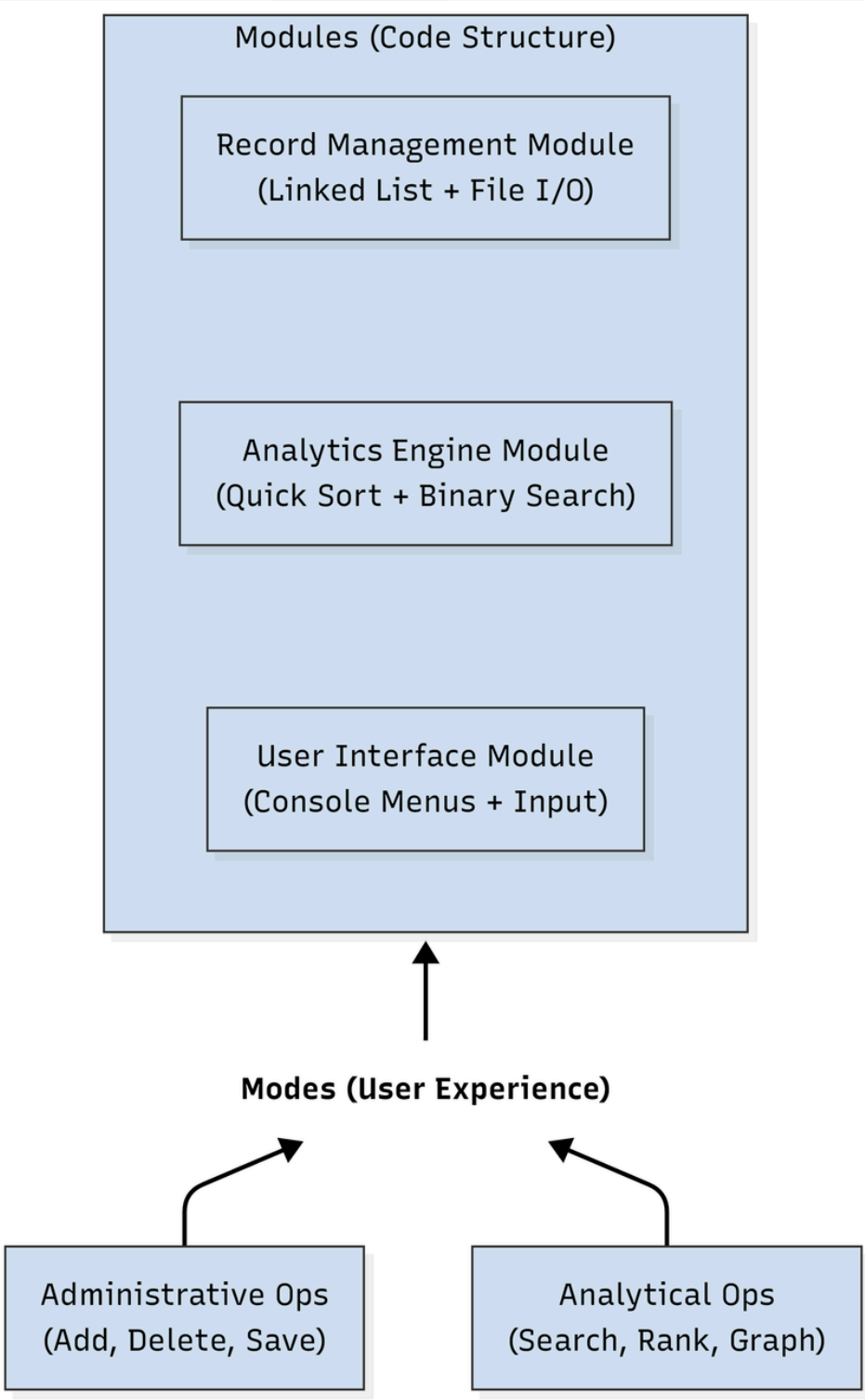
Implementation

System Architecture

The system architecture comprises three core modules using C:

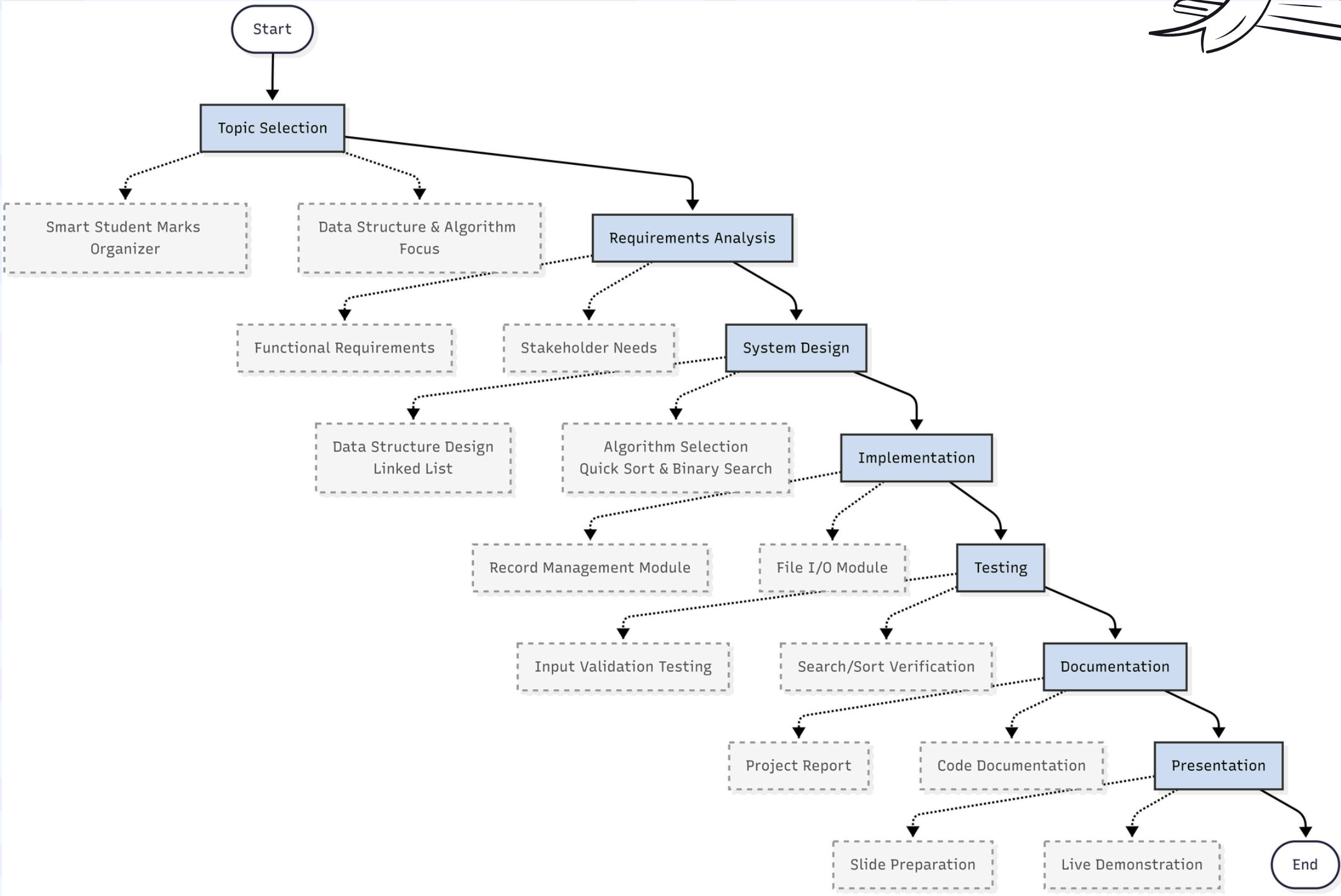
- ✔ Record Management Module
- ✔ Analytics Engine Module
- ✔ User Interface Module

Tools: ANSI C Language, Singly Linked Lists, File I/O.



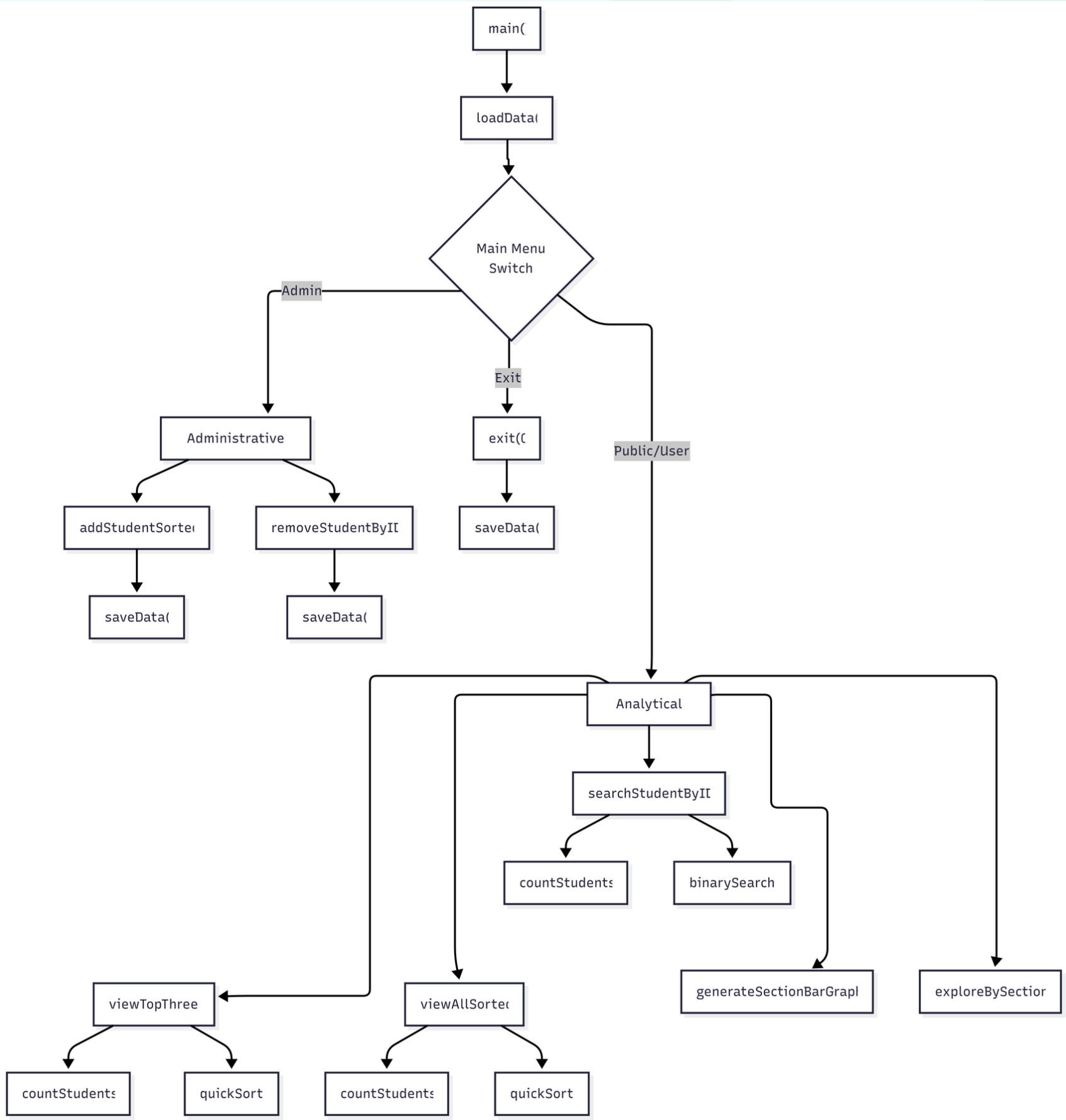
Implementation

Work Flow



Implementation

Function Flow



All our important functions

- `addStudentSorted()`
- `viewTopThree()`
- `exploreBySection()`
- `generateSectionBarGraph()`
- `searchStudentByID()`
- `removeStudentByID()`
- `viewAllSorted()`



addStudentSorted() Function



Main Steps:

- Create a new student node
- Validate inputs (unique ID, valid name, marks 0–100)
- Calculate total marks
- Insert into list (sorted by ID)
- Save updated data

Code (Page 21 - 24)



```
76 void addStudentSorted()
77 {
78     struct Student *newnode = (struct Student *)malloc(sizeof(struct Student));
79     struct Student *temp;
80     int exists, valid;
81
82     printf("\n--- [ ADD NEW CSE STUDENT ] ---\n");
83
84     while (1)
85     {
86         printf("Enter Student ID: ");
87         if (scanf("%d", &newnode->id) != 1)
88         {
89             while (getchar() != '\n')
90                 ;
91             printf("Invalid input! Please enter a number.\n");
92             continue;
93         }
94
95         temp = start;
96         exists = 0;
97         while (temp != NULL)
98         {
99             if (temp->id == newnode->id)
100             {
101                 exists = 1;
102                 break;
103             }
104             temp = temp->next;
105         }
106
107         if (exists)
```



```
107     if (exists)
108         printf("Error: This ID already exists! Please enter a unique ID.\n");
109     else
110         break;
111 }
112
113 while (1)
114 {
115     printf("Enter Student Name: ");
116     scanf(" %[^\n]", newnode->name);
117
118     valid = 1;
119     for (int i = 0; newnode->name[i] != '\0'; i++)
120     {
121         if (!isalpha(newnode->name[i]) && !isspace(newnode->name[i]) && newnode->name[i] != '.')
122         {
123             valid = 0;
124             break;
125         }
126     }
127
128     if (!valid)
129         printf("Error: Name must contain only letters and spaces (Ex: Mohammad).\n");
130     else
131         break;
132 }
133
134 printf("Enter Section (Ex: 68_A, 68_B): ");
135 scanf("%14s", newnode->section);
136
```



```
137 const char *course_names[] =
138 {
139     "Algorithms(CSE213)", "Algorithms Lab(CSE214)", "Engineering Mathematics(MAT211)",
140     "Discrete Mathematics(CSE212)", "Bangladesh Studies(BNS101)"};
141
142 printf("\n-- Enter Course Marks (0 - 100) --\n");
143 for (int c = 0; c < 5; c++)
144 {
145     while (1)
146     {
147         printf("%s: ", course_names[c]);
148         if (scanf("%d", &newnode->courses[c]) != 1)
149         {
150             while (getchar() != '\n')
151                 ;
152             printf("Invalid input! Please enter a number.\n");
153             continue;
154         }
155
156         if (newnode->courses[c] < 0 || newnode->courses[c] > 100)
157         {
158             printf("Error: Marks must be between 0 and 100. Please try again.\n");
159         }
160         else
161             break;
162     }
163 }
164
165 newnode->total_marks = newnode->courses[0] + newnode->courses[1] + newnode->courses[2] + newnode->courses[3] + newnode->courses[4];
166
167 newnode->next = NULL;
```



```
168
169  if (start == NULL || start->id >= newnode->id)
170  {
171      newnode->next = start;
172      start = newnode;
173  }
174  else
175  {
176      struct Student *current = start;
177      while (current->next != NULL && current->next->id < newnode->id)
178      {
179          current = current->next;
180      }
181      newnode->next = current->next;
182      current->next = newnode;
183  }
184
185  printf("\n Student added successfully!\n");
186
187  saveData();
188  }
```

viewTopThree() Function



Main Steps:

- Count students; exit if none
- Store list into an array
- Sort array by marks (quickSort)
- Print top 3 (or less) students
- Free memory

Code



```

320 void viewTopThree(){
321     int n = countStudents();
322     if (n == 0)
323     {
324         printf("\nNo student records available.\n");
325         return;
326     }
327
328     struct Student **arr = malloc(n * sizeof(struct Student *));
329     struct Student *curr = start;
330     for (int i = 0; i < n; i++)
331     {
332         arr[i] = curr;
333         curr = curr->next;
334     }
335
336     printf("\n--- [ HALL OF FAME: TOP 3 CSE STUDENTS ] ---\n");
337
338     quickSort(arr, 0, n - 1);
339
340     printf("%-5s | %-6s | %-20s | %-7s | %-5s\n", "RANK", "ID", "NAME", "SECTION", "TOTAL");
341     printf("-----\n");
342
343     int limit = (n < 3) ? n : 3;
344     for (int i = 0; i < limit; i++)
345     {
346         printf(" #%-3d | %-6d | %-20s | %-7s | %-5d\n",
347             i + 1, arr[i]->id, arr[i]->name, arr[i]->section, arr[i]->total_marks);
348     }
349     printf("-----\n");
350     free(arr);
351 }

```

exploreBySection() Function



Main Steps:

- Take section input
- Traverse list and match section
- Print matching students
- If none found, show message

Code



```

190 void exploreBySection()
191 {
192     char sec[15];
193     printf("\n--- [ CSE SECTION EXPLORER ] ---\n");
194     printf("Enter Section to view ( Ex: 68_A): ");
195     scanf("%14s", sec);
196
197     printf("\nFetching data for Section %s...\n", sec);
198     printf("--- [ SECTION %s: COURSE WISE LIST ] ---\n", sec);
199
200     printf("%-6s | %-18s | %-3s | %-3s | %-3s | %-3s | %-3s | %-5s\n", "ID", "Name", "Alg", "Alg_L", "EM", "DM", "BNS", "Total");
201     printf("-----\n");
202
203     struct Student *i = start;
204     int found = 0;
205     while (i != NULL)
206     {
207         if (strcmp(i->section, sec) == 0)
208         {
209             printf("%-6d | %-18s | %-3d | %-5d | %-3d | %-3d | %-3d | %-5d\n",
210                 i->id, i->name, i->courses[0], i->courses[1], i->courses[2], i->courses[3], i->courses[4], i->total_marks);
211             found = 1;
212         }
213         i = i->next;
214     }
215
216     if (!found)
217         printf("No students found in Section %s.\n", sec);
218     printf("-----\n");
219 }

```

generateSectionBarGraph() Function



- Take section input
- Initialize sum array and student count
- Traverse linked list
- Match section and accumulate marks + increment count
- If no students found, print message and return
- Print graph header
- Loop through courses
- Calculate average for each course
- Convert average to bar length
- Print bar graph using #

Code (30 - 31)



```

391 void generateSectionBarGraph() {
392     char sec[15];
393     printf("\n--- [ SECTION AVERAGE BAR GRAPH ] ---\n");
394     printf("Enter Section to analyze (Ex: 68_A): ");
395     scanf("%14s", sec);
396
397     int sum[5] = {0, 0, 0, 0, 0};
398     int count = 0;
399
400     struct Student* i = start;
401     while (i != NULL) {
402         if (strcmp(i->section, sec) == 0) {
403             for (int c = 0; c < 5; c++) {
404                 sum[c] += i->courses[c];
405             }
406             count++;
407         }
408         i = i->next;
409     }
410
411     if (count == 0) {
412         printf("\nNo students found in Section %s to generate a graph.\n", sec);
413         return;
414     }
415
416     printf("\nGenerating Bar Graph for Section %s (Total Students: %d)\n\n", sec, count);
417
418     const char* course_names[] =
419     {
420         "Algorithms(CSE213)           ",
421         "Algorithms Lab(CSE214)        ",
422         "Engineering Mathematics(MAT211) ",

```




```
423         "Discrete Mathematics(CSE212)        ",
424         "Bangladesh Studies(BNS101)         "};
425
426     for (int c = 0; c < 5; c++) {
427         float avg = (float)sum[c] / count;
428         printf("%s [Avg: %5.1f] : ", course_names[c], avg);
429
430         int bars = (int)(avg / 2);
431         for (int b = 0; b < bars; b++) {
432             printf("#");
433         }
434         printf("\n");
435     }
436 }
```

searchStudentByID() Function



- Take student ID input
- Get total number of students
- If empty, print message and return
- Convert linked list to array
- Perform binary search on array
- If found, print student details
- If not found, print message
- Free allocated memory

Code (33 - 34)

```

349 void searchStudentByID() {
350     int target_id;
351     printf("\n--- [ SEARCH STUDENT ] ---\n");
352     printf("Enter Student ID to search: ");
353     scanf("%d", &target_id);
354
355     int n = countStudents();
356     if (n == 0) {
357         printf("\nDatabase is empty.\n");
358         return;
359     }
360
361     struct Student** arr = malloc(n * sizeof(struct Student*));
362     struct Student* curr = start;
363     for (int i = 0; i < n; i++) {
364         arr[i] = curr;
365         curr = curr->next;
366     }
367
368     int index = binarySearch(arr, 0, n - 1, target_id);
369
370     if (index != -1) {
371         printf("\n--- [ STUDENT RECORD FOUND ] ---\n");
372         printf("ID      : %d\n", arr[index]->id);
373         printf("Name    : %s\n", arr[index]->name);
374         printf("Section : %s\n", arr[index]->section);
375         printf("-----\n");
376         printf("Algorithms (CSE213)      : %d\n", arr[index]->courses[0]);
377         printf("Algorithms Lab (CSE214)  : %d\n", arr[index]->courses[1]);
378         printf("Engineering Math (MAT211) : %d\n", arr[index]->courses[2]);
379         printf("Discrete Math (CSE212)   : %d\n", arr[index]->courses[3]);
380         printf("Bangladesh Studies (BNS101): %d\n", arr[index]->courses[4]);

```





```
381     printf("-----\n");
382     printf("TOTAL MARKS           : %d\n", arr[index]->total_marks);
383     printf("-----\n");
384 } else {
385     printf("\nStudent not found!\n");
386 }
387
388 free(arr);
389 }
```

removeStudentByID() Function



- Take student ID input with validation
- If list is empty, print message and return
- Initialize pointers (temp and prev)
- Check if first node matches target ID
- If matched, update start, delete node, save data, return
- Traverse list to find target node
- If not found, print message and return
- Update links to remove node (prev->next = temp->next)
- Free the node memory
- Print success message and save data

Code (36 - 37)

```
192 void removeStudentByID() {
193     int target_id;
194     printf("\n--- [ REMOVE STUDENT RECORD ] ---\n");
195     printf("Enter Student ID to remove: ");
196
197     if (scanf("%d", &target_id) != 1) {
198         while (getchar() != '\n');
199         printf("Invalid input! Please enter a valid ID number.\n");
200         return;
201     }
202
203     if (start == NULL) {
204         printf("\nDatabase is empty. No students to remove.\n");
205         return;
206     }
207
208     struct Student* temp = start;
209     struct Student* prev = NULL;
210
211     if (temp != NULL && temp->id == target_id) {
212         start = temp->next;
213         free(temp);
214         printf("\n Student with ID %d has been removed successfully!\n", target_id);
215         saveData();
216         return;
217     }
218
219     while (temp != NULL && temp->id != target_id) {
220         prev = temp;
221         temp = temp->next;
222     }
223 }
```





```
224 ✓ if (temp == NULL) {  
225     printf("\n Student with ID %d not found in the database.\n", target_id);  
226     return;  
227 }  
228  
229 prev->next = temp->next;  
230 free(temp);  
231  
● 232 printf("\n Student with ID %d has been removed successfully!\n", target_id);  
233 saveData();  
234 }
```

viewAllSorted() Function



- Get total number of students
- If empty, print message and return
- Convert linked list to array
- Print ranking header
- Sort array using quickSort()
- Print table headings
- Loop through all students and print ranked data
- Print closing line
- Free allocated memory

Code



```
307 void viewAllSorted() {
308     int n = countStudents();
309     if (n == 0) {
310         printf("\nNo student records available.\n");
311         return;
312     }
313
314     struct Student** arr = malloc(n * sizeof(struct Student*));
315     struct Student* curr = start;
316     for (int i = 0; i < n; i++) {
317         arr[i] = curr;
318         curr = curr->next;
319     }
320
321     printf("\n--- [ FULL RANKING: ALL CSE STUDENTS ] ---\n");
322
323     quickSort(arr, 0, n - 1);
324
325     printf("%-5s | %-6s | %-20s | %-7s | %-5s\n", "RANK", "ID", "NAME", "SECTION", "TOTAL");
326     printf("-----\n");
327
328     for (int i = 0; i < n; i++) {
329         printf(" #%-3d | %-6d | %-20s | %-7s | %-5d\n",
330             i + 1, arr[i]->id, arr[i]->name, arr[i]->section, arr[i]->total_marks);
331     }
332     printf("-----\n");
333     free(arr);
334 }
```

Results and Discussion

Results

- **Add Student Record Testing:** ID uniqueness, name and marks validation, sorted insertion by ID, and data persistence all function correctly.
- **Quick Sort / Top 3 Ranking Testing:** Top 3 students are accurately identified with correct descending order; edge cases (0–3+ students) handled properly.
- **Binary Search Testing:** Efficiently retrieves existing students, correctly returns not found for invalid IDs, and maintains $O(\log n)$ performance.
- **Section Explorer Testing:** Successfully filters students by section, displays complete course data, and handles invalid sections gracefully.
- **Bar Graph Generation Testing:** Accurately computes course averages and generates proportional visual bars, with proper handling of empty sections.
- **Remove Student Testing:** Correctly deletes nodes from any position, ensures memory is freed, and updates persistent storage reliably.



Results and Discussion

Discussion

- **Algorithm Efficiency & Optimization**
- **Data Structure Implementation Success**
- **Memory Management Validation**
- **Error Handling & Robustness**
- **Data Persistence & Integrity**
- **Real-World Applicability**





Advantages and Disadvantages

Advantages

- Low Resource Usage
- Persistent Data Storage
- Efficient Sorting & Searching (Quick Sort + Binary Search)
- Strong Input Validation
- Section-wise Analysis & Visualization
- Modular and Structured Code

Disadvantages

- No Graphical User Interface (CLI Based)
- Limited Security (No Authentication)
- Linked List Traversal Overhead
- No Update/Edit Feature
- Not Fully Scalable for Large Data
- Basic Visualization (Text-Based Only)

Conclusion



Summary

The Smart Student Marks Organizer successfully provides a functional and efficient student record management system using linked lists in C. It supports sorted insertion, fast searching (Binary Search), and ranking (Quick Sort), along with section-wise analysis and visualization. Key outcomes include reliable data persistence, strong input validation, proper memory management, and a modular structure—effectively bridging data structure concepts with practical academic applications.

Future Scope

- GUI Implementation
- Hash-Based Searching
- Advanced Visualization
- Edit/Update Feature
- Database Integration
- User Authentication



Thank You

Any Question?

